Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Множества на основе битовых полей»

Выполнил:
студент группы 3825Б1ФИип2
Лумпов И.Л

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 5
3. Руководство программиста 6
3.1. Описание структуры программы 6
3.2. Описание структур данных 6
3.3. Описание алгоритмов 7
4. Результаты экспериментов и тестирования 9
Заключение 10
Литература 11
Приложение 12

Введение

Программа представляет собой реализацию структур данных для хранения множеств с использованием битовых полей на языке C++. В рамках работы реализованы два класса: TBitField (битовое поле) и TSet (множество). Программа не имеет графического интерфейса, так как является библиотекой. Проверка корректности работы реализованных структур данных осуществляется с помощью фреймворка для автоматического тестирования Google Test.

1. Постановка задачи

Требуется разработать программу на языке C++ в виде библиотеки классов, реализующую структуру данных для хранения множеств на основе битовых полей.

Задачи:

1. Реализация класса битового поля TBitField согласно заданному интерфейсу.
2. Реализация класса множества TSet согласно заданному интерфейсу.
3. Написание и обеспечение работоспособности тестов с использованием Google Test, покрывающих основной функционал и краевые случаи.
4. Обеспечение обработки некорректного поведения программы (проброс исключений).
5. Операции с множествами допустимы только в пределах одного универсума (мощности множеств должны совпадать).

2. Руководство пользователя

Так как программа является библиотекой, а не консольным приложением, руководство пользователя описывает процесс запуска автоматических тестов.

1. Для запуска необходимо открыть решение в Visual Studio, выбрать в верхнем меню Тест (Test) -> Обозреватель тестов (Test Explorer).

2. В открывшемся окне нажать кнопку «Запустить все тесты» (Run All Tests).

3. В окне обозревателя тестов отобразятся все группы тестов. Зеленые галочки означают, что тест пройден успешно, красные крестики — что тест провален, в этом случае под тестом выводится сообщение об ошибке с указанием ожидаемого и фактического значений.

3. Руководство программиста

3.1. Описание структуры программы

Программа состоит из трех основных модулей:

BitField — статическая библиотека, содержащая заголовочные файлы (tbitfield.h, tset.h) и файлы реализации (tbitfield.cpp, tset.cpp). Содержит классы TBitField и TSet.

BitFieldTests — проект с тестами на базе Google Test. Содержит файлы test_bitfield.cpp и test_tset.cpp. Линкуется с проектом BitField.

Файлы заголовков подключаются из общей папки includes с помощью макроса $(SolutionDir)..\..\includes\Lab1.

3.2. Описание структур данных

typedef unsigned int TELEM — тип данных для хранения элементов битового поля (обычно 32-битное беззнаковое целое).

Глобальные константы в тестах: генерируются случайным образом длины от 1 до 200 для проверки свойств идемпотентности, коммутативности и двойного отрицания.

class TBitField:

- int BitLen — длина битового поля (количество битов);
- TELEM* pMem — динамический массив для хранения битов;
- int MemLen — количество элементов в массиве pMem (вычисляется как (BitLen + 31) / 32);
- int GetMemIndex(const int n) — получение индекса элемента в массиве pMem для бита n;
- TELEM GetMemMask(const int n) — получение битовой маски для бита n;
- TBitField(int len) — конструктор (бросает std::invalid_argument, если len <= 0);
- TBitField(const TBitField& bf) — конструктор копирования;
- ~TBitField() — деструктор;
- int GetLength() const — получить длину;
- void SetBit(const int n) — установить бит (бросает std::out_of_range);
- void ClrBit(const int n) — очистить бит (бросает std::out_of_range);
- int GetBit(const int n) const — получить значение бита (бросает std::out_of_range);
- operator==, operator!= — операции сравнения;
- operator= — оператор присваивания;

- operator|, operator& — логические операции (бросают std::invalid_argument, если длины полей разные);
- operator~ — отрицание;
- дружественные операторы >> и << — ввод/вывод битового поля в виде строки из 0 и 1.

class TSet:

- int MaxPower — максимальная мощность множества (размер универсума);
- TBitField BitField — характеристический вектор множества, использует композицию, а не наследование от TBitField;
- TSet(int mp) — конструктор (бросает std::invalid_argument);
- TSet(const TSet &s) — конструктор копирования;
- TSet(const TBitField &bf) — конструктор преобразования;
- operator TBitField() — преобразование к битовому полю;
- int GetMaxPower() const — получить мощность;
- void InsElem(const int Elem) — добавить элемент (бросает std::out_of_range);
- void DelElem(const int Elem) — удалить элемент (бросает std::out_of_range);
- int IsMember(const int Elem) const — проверка принадлежности (бросает std::out_of_range);
- operator==, operator!=, operator= — сравнение и присваивание;
- operator+ (с элементом и с множеством), operator- (с элементом), operator* (пересечение), operator~ (дополнение);
- дружественные операторы >> и << — ввод/вывод в формате { 1, 3, 5 }.

3.3. Описание алгоритмов

1. Работа с битовым полем:

Битовое поле хранится в виде массива беззнаковых целых чисел. Каждый бит с номером n находится в элементе pMem[n / 32] под номером n % 32 (считая с младшего). Установка бита выполняется с помощью побитового ИЛИ с маской (1 << (n % 32)), очистка — с помощью побитового И с инвертированной маской.

2. Операции над битовыми полями разной длины.

В отличие от множеств, битовые поля могут иметь разную длину. При выполнении операций | (ИЛИ) и & (И) результирующее поле имеет максимальную длину из двух операндов. Если один операнд короче, его недостающие старшие элементы считаются равными нулю. Это позволяет корректно объединять маски разной длины.

3. Безопасный ввод битового поля.

При чтении битового поля из потока (operator>>) данные сначала считываются во временный объект. Если в процессе чтения возникает ошибка, временный объект удаляется, а

исходный объект сохраняет свои прежние значения. Это предотвращает частичное обновление состояния объекта при некорректном вводе.

4. Операции над множествами:

Множество представляется как битовое поле, где i-й бит равен 1, если элемент i принадлежит множеству.

Объединение (+) выполняется как побитовое ИЛИ характеристических векторов. Пересечение (*) — как побитовое И. Дополнение (~) — как побитовая инверсия.

Перед выполнением операций над двумя множествами проверяется равенство их мощностей (MaxPower). Если мощности различаются, генерируется исключение std::invalid_argument, так как множества принадлежат разным универсумам.

5. Обработка исключений:

Для предотвращения неопределенного поведения при выходе за границы массива в методах доступа к битам и элементам множеств добавлены проверки. При некорректных индексах генерируется std::out_of_range. При попытке создать поле/множество неположительной длины генерируется std::invalid_argument.

6. Тестирование:

Тесты разделены на группы: Smoke (базовые проверки), Functional (основной функционал), EdgeCases (краевые случаи) и Exceptions (проверка проброса исключений с помощью EXPECT_THROW).

Отдельно реализованы тесты на случайных данных (Random), которые проверяют выполнение математических инвариантов: идемпотентность (a | a == a, a + a == a), коммутативность (a | b == b | a, a + b == b + a), двойное отрицание (~~a == a) и законы де Моргана (~(a | b) == ~a & ~b).

4. Результаты экспериментов и тестирования

Тестирование проводилось с помощью фреймворка Google Test в среде Visual Studio. Было разработано 28 тестов, покрывающих все заявленные методы классов.

Тест 1 (Базовые операции).

Создается битовое поле длины 64. Устанавливаются биты 5, 31 и 63. Проверяется, что метод GetBit возвращает 1 для этих битов и 0 для остальных. Тест пройден.

Тест 2 (Обработка исключений).

Проверяется, что при вызове TBitField bf(0) генерируется исключение std::invalid_argument. При вызове bf.SetBit(-1) и bf.SetBit(100) для поля длины 10 генерируется std::out_of_range. При попытке выполнить a | b для полей разной длины генерируется std::invalid_argument. Все исключения перехвачены макросом EXPECT_THROW. Тест пройден.

Тест 3 (Множества и разные универсумы).

Создаются два множества TSet a(10) и TSet b(20). Проверяется, что попытка выполнить a + b или a * b приводит к исключению std::invalid_argument. Тест пройден.

Тест 4 (Случайные данные).

В цикле (100 итераций) генерируются случайные битовые поля и множества случайной длины (от 1 до 200). Проверяется, что для них выполняются законы де Моргана и другие инварианты. Все 100 итераций пройдены без ошибок.

Тест 5 (Безопасный ввод и разные длины).

Проверяется, что при вводе строки с ошибкой (например, 1 0 X 1 1) битовое поле сохраняет свое предыдущее состояние. Также проверяется, что операция a | b для полей длиной 8 и 16 бит возвращает поле длиной 16 бит, корректно объединяя данные (старшие биты короткого поля считаются нулями).

Общий результат: Все 28 тестов, включая Smoke, Functional, EdgeCases, Exceptions и Random, выполнены успешно.

Заключение

В ходе выполнения лабораторной работы были достигнуты следующие результаты:

- Реализован класс TBitField, обеспечивающий компактное хранение битовых масок и поддерживающий все базовые побитовые операции и поддерживающий операции над полями разной длины.

- Реализован класс TSet на основе композиции с TBitField, представляющий собой множество на основе битового поля и поддерживающий основные теоретико-множественные операции.

- Внедрена обработка ошибок с использованием механизма исключений C++ (std::invalid_argument, std::out_of_range), что позволяет однозначно определять корректность работы программы извне.

- Реализована проверка совпадения универсумов при операциях над множествами.

- Обеспечен безопасный ввод битовых полей: при ошибке ввода объект сохраняет свое исходное состояние.

- Написан набор из 28 автотестов с использованием фреймворка Google Test, покрывающий основной функционал, граничные случаи, исключительные ситуации, а также тесты на случайных данных для проверки математических свойств операций.

Литература

Список источников, используемых при выполнении работы (если есть).

1. Страуструп Бьерн Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.
2. Сайт cppreference: https://en.cppreference.com/

Приложение